

└─ /opt/scripts/	← old backup scripts (kept as fallback)
└─ v2/	← new backup scripts
└─ frappe_backup.sh	← full backup (DB + files + config)
└─ frappe_db_backup.sh	← quick database-only backup
└─ frappe_backup_verify.sh	← checks backups are complete & fresh
└─ frappe_restore.sh	← restore a full backup
└─ /opt/backups/frappe/	← data backups (shared)
└─ 20260712_020000/	← one folder per full backup
└─ *-database.sql.gz	← database dump
└─ *-files.tgz	← public files
└─ *-private-files.tgz	← private files / attachments
└─ site_config.json, .env, docker-compose.yml	
└─ backup_manifest.txt	
└─ db-only/	← 6-hourly database dumps

2. Prerequisites

- Ubuntu Server 24.04 with Docker Engine + Docker Compose v2 (`docker compose` , not `docker-compose`).
- The ERPNext stack deployed at `/opt/sspl-erp/` with a `docker-compose.yml` and a `.env` file containing `MARIADB_ROOT_PASSWORD` .
- Root / sudo access on the server.
- Your Frappe **site name**. To find it:

```
docker compose -f /opt/sspl-erp/docker-compose.yml exec backend ls sites/
```

The site name is the folder that is not `apps.txt` , `assets` , or `common_site_config.json` — for example `192.168.225.135` .

3. Fresh server — one-command installation

For a **new server** (nothing deployed yet), one script performs the entire production deployment — the whole *SSPL ERP Production Deployment Guide* — plus Parts 1–3 of this toolkit, in a single run:

```
git clone https://github.com/santhosh3279/sspl_installation.git
cd "sspl_installation/Production Installation"
chmod +x install_sspl_erp.sh
./install_sspl_erp.sh
```

All questions are asked **up front** — server IP, HTTP port (default **80**), MariaDB root password, ERPNext Administrator password, which components to install, and the admin panel login. After the final *Proceed?* it runs unattended for 10–20 minutes:

- **ERP stack** — clones `frappe_docker` , writes `.env` and the pdf-fix override, generates `/opt/sspl-erp/docker-compose.yml` , pulls the `sspl-erpnext` image from GHCR, starts all containers, and waits until the database and backend are actually ready.
- **Site creation (one-time)** — creates the ERPNext site named after the server IP and installs **every app shipped in the image** automatically (`erpnext`, `india_compliance`, `ssplbilling`, `printer_server_configuration`, `frappe_whatsapp`, ...), sets it as the default site, and repairs the MariaDB grants.
- **Part 1** — backup system with cron jobs (`/opt/scripts/v2/`).
- **Part 2** — update & rollback scripts with the site name pre-filled (`/opt/sspl-erp/v2/`).
- **Part 3** — web admin panel with HTTPS on port 8090 (`/opt/sspl-admin/`).
- **Extras** — Docker auto-start on reboot, docker group membership, and passwordless sudo shortcuts for clear-RAM and manual backup.

Already installed? It refuses to run. If ERPNext is already deployed and running on the server, the installer prints *“already installed”* and exits without changing anything — use the update tools instead (sections 9 and 14). To deliberately re-run, e.g. to finish a half-completed install, use `SSPL_FORCE=1 ./install_sspl_erp.sh` ; a forced re-run is safe — the existing site is never re-created, and existing `.env` files are backed up before being rewritten.

Won't double-schedule backups. Before installing the v2 backup cron jobs, the installer checks the crontab for an existing entry pointing at `/opt/scripts/` outside `/opt/scripts/v2/` — e.g. an older backup script wired up by hand (the

“Santhosh additions” in this guide's history). If one is found, the v2 scripts are still installed but their cron jobs are **not** scheduled automatically, and the conflicting line is printed so you can switch over manually with `sudo crontab -e` (see section 12 for the v2 schedule).

Requirements: Ubuntu Server with Docker installed and a static LAN IP. If you install components with this script, you can skip the manual installation steps in sections 4, 8 and 13.

4. Part 1 — Backup system: installation

Step 1 — Copy the files to the server

```
# From the repo, copy either the folder or the tarball to the server, then:
tar -xzf frappe_backup_system.tar.gz
cd frappe_backup_system          # or: cd Backup/frappe_backup_system
```

Step 2 — Run the installer

```
chmod +x setup_frappe_backups.sh
sudo ./setup_frappe_backups.sh
```

The installer will:

1. Create `/opt/backups/frappe/` (permissions restricted to root — backups contain credentials).
2. Install the four scripts to `/opt/scripts/v2/` and make them executable — older scripts directly in `/opt/scripts/` are not touched.
3. **Ask for your site name** and write it into the backup, db-backup, and restore scripts.
4. Offer to install the cron jobs (see section 12 for the schedule).
5. Offer to run a test backup (answer `yes` the first time).

If the old backup scripts are already scheduled in cron: installing the new cron jobs too means every backup runs twice. Either answer `no` to the cron question while you test the v2 scripts manually, or say `yes` and then remove the old `/opt/scripts/frappe_*.sh` lines with `sudo crontab -e` once you are happy with v2.

Step 3 — Verify the test backup

```
sudo /opt/scripts/v2/frappe_backup_verify.sh
```

All four lines must say **OK**: database, public files, private files, and config. The script exits with an error if anything is missing.

5. Part 1 — Backup system: daily use

Once installed, backups run automatically from cron. Manual commands:

Task	Command
Full backup now (DB + files + config)	<code>sudo /opt/scripts/v2/frappe_backup.sh</code>
Quick database-only backup	<code>sudo /opt/scripts/v2/frappe_db_backup.sh</code>
Check backups are complete & recent	<code>sudo /opt/scripts/v2/frappe_backup_verify.sh</code>
See what is stored	<code>ls -lh /opt/backups/frappe/</code>
Watch the backup log	<code>tail -f /var/log/frappe_backup_v2.log</code>

Retention is automatic: full backups are kept **30 days**, database-only dumps **14 days**. To change this, edit `RETENTION_DAYS` at the top of the respective script in `/opt/scripts/v2/`.

Good habit: run a manual full backup before any risky change (new app install, configuration change, server maintenance) — and always before an update, although the update script also does this automatically.

6. Part 1 — Restoring from a backup

Warning: a restore **overwrites** the site's current database and files with the backup contents. Anything entered after the backup was taken is lost.

Step 1 — Pick a backup

```
ls /opt/backups/frappe/  
# e.g. 20260712_020000
```

Step 2 — Get the MariaDB root password

```
grep MARIADB_ROOT_PASSWORD /opt/sspl-erp/.env
```

Step 3 — Run the restore

```
sudo /opt/scripts/v2/frappe_restore.sh /opt/backups/frappe/20260712_020000
```

The script will:

1. Ask for the site name (press Enter to accept the configured default).
2. Ask for the MariaDB root password (input is hidden).
3. Ask for a final `yes` confirmation.
4. Restore the database, public files, and private files in one operation.
5. Run `bench migrate` (in case app versions differ from the backup) and restart all services.

Afterwards, log in to ERPNext and verify recent documents, attachments, and print formats.

7. Part 1 — Optional cloud upload (rclone)

By default backups stay on the server's own disk. To survive a disk failure, upload them to cloud storage. Full provider walkthrough: see *Rclone_Configuration_Guide.docx*.

Step 1 — Install and configure rclone (one time)

```
sudo apt install rclone  
rclone config      # create a remote, e.g. named "gdrive"
```

Step 2 — Point the backup script at the remote

Edit `/opt/scripts/v2/frappe_backup.sh` and set the variable near the top:

```
RCLONE_REMOTE="gdrive:frappe-backups"
```

Every full backup is then uploaded automatically after it completes. If the upload fails, the backup still exists locally and a warning is written to the log.

Step 3 — Test it

```
sudo /opt/scripts/v2/frappe_backup.sh  
rclone ls gdrive:frappe-backups | head
```

8. Part 2 — Update & rollback system: installation

Step 1 — Copy the four scripts

Copy these from `Production Installation/update` and `rollback/` into `/opt/sspl-erp/v2/` (create it with `sudo mkdir -p /opt/sspl-erp/v2`). Older `sspl-erp-*.sh` scripts directly in `/opt/sspl-erp/` stay untouched as a fallback; both versions share the same `image-backups/` snapshots.

- `sspl-erp-common.sh` — shared configuration (sourced by the others, not run directly)
- `sspl-erp-update-with-rollback.sh`
- `sspl-erp-rollback.sh`
- `sspl-erp-backup-manager.sh`

Step 2 — Set the site name

Edit `/opt/sspl-erp/v2/sspl-erp-common.sh` and set the variable at the top (this is the **only** place it needs changing):

```
SITE_NAME="192.168.225.135"
```

Step 3 — Make the scripts executable

```
cd /opt/sspl-erp/v2
chmod +x sspl-erp-update-with-rollback.sh sspl-erp-rollback.sh sspl-erp-backup-manager.sh
```

Note: the update script calls `/opt/scripts/v2/frappe_backup.sh`, so install Part 1 first.

9. Part 2 — Updating the system

When a new image has been published to GHCR (`ghcr.io/santhosh3279/sspl-erpNext`):

```
cd /opt/sspl-erp/v2
./sspl-erp-update-with-rollback.sh
```

What it does, in order:

1. Runs a **full Frappe data backup** (database + files). If this fails you are asked whether to continue.
2. Snapshots the current Docker images to `/opt/sspl-erp/image-backups/backup_TIMESTAMP.tar`.
3. Stops all services, prunes unused Docker resources, and pulls the latest images.
4. Starts services and **waits until the database and backend are actually ready** (up to 3 minutes).
5. Repairs MariaDB user grants, runs `bench migrate`, clears the cache.
6. Prints the new version and deletes image snapshots older than the 3 most recent.

If any step fails, the script stops immediately and prints rollback instructions.

Verify after every update

```
docker compose ps
docker compose logs --tail=50
docker compose exec backend bench version
docker compose exec backend bench --site 192.168.225.135 doctor
```

Then log in and check the key business flows (e.g. create a draft invoice, open a report, test printing).

10. Part 2 — Rolling back an update

If the system misbehaves after an update:

```
cd /opt/sspl-erp/v2
./sspl-erp-rollback.sh
```

The script shows the most recent image snapshot (size and date), asks for confirmation, stops the services, loads the saved images, restarts, repairs DB grants, and clears the cache.

Rolling back to a specific older snapshot

```
./sspl-erp-backup-manager.sh list
BACKUP_FILE=/opt/sspl-erp/image-backups/backup_20260701_090000.tar ./sspl-erp-rollback.sh
```

Important: rollback restores the Docker **images** only — not the database. If the failed update's migration changed the database structure, also restore the data backup that the update script took automatically (section 6). Its timestamp is printed at the start of the update.

11. Part 2 — Managing image backups

Image snapshots are several GB each. The update script already keeps only the last 3; the manager gives manual control:

Task	Command
List snapshots (marks the latest)	<code>./sspl-erp-backup-manager.sh list</code>
Keep newest 5, delete the rest	<code>./sspl-erp-backup-manager.sh clean 5</code>
Same, without confirmation (for cron)	<code>./sspl-erp-backup-manager.sh clean 5 --yes</code>
Delete all snapshots	<code>./sspl-erp-backup-manager.sh delete-all</code>
Check disk space	<code>df -h /opt/sspl-erp</code>

Note: when run from cron there is no terminal to confirm on, so `--yes` is required — without it the command exits with an error instead of deleting.

12. Scheduled jobs summary

Installed by the Part 1 setup script (view with `sudo crontab -l`):

Schedule	Job	Log file
Daily, 02:00	Full backup — <code>frappe_backup.sh</code>	<code>/var/log/frappe_backup_v2.log</code>
Every 6 hours	DB-only backup — <code>frappe_db_backup.sh</code>	<code>/var/log/frappe_db_backup_v2.log</code>
Sunday, 03:00	Backup verification — <code>frappe_backup_verify.sh</code>	<code>/var/log/frappe_backup_verify_v2.log</code>

Optional — weekly cleanup of image snapshots (add with `sudo crontab -e`):

```
0 2 * * 0 /opt/sspl-erp/v2/sspl-erp-backup-manager.sh clean 5 --yes >> /var/log/sspl-erp-backup-clean.log 2>&1
```

13. Part 3 — Web admin panel

A browser interface for everything above: **install the whole system from the browser**, then run backups, updates, and rollbacks with one click, watch the script output live, browse and download backup files, monitor CPU / memory / disk / container health, clear RAM caches, and upload backup files to the server.

Installation

```
git clone https://github.com/santhosh3279/sspl_installation.git
cd sspl_installation/"Admin Panel"
chmod +x setup_admin_panel.sh
./setup_admin_panel.sh
```

The installer asks for an admin username, password, port (default **8090**), and the server IP for the HTTPS certificate. It installs a Python environment to `/opt/sspl-admin/`, records the repo location (`repo_dir` in its config), and starts a systemd service (`sspl-admin`) that survives reboots.

The panel can install everything else — Parts 1 and 2 do *not* need to be in place first. Keep the git checkout on the server (the Setup switches run the installers from it; `git pull` there keeps them current).

Panel-first setup

Open `https://<server-ip>:8090` and log in. The certificate is self-signed (a LAN IP cannot get a public certificate), so the browser warns the **first** time — click *Advanced* → *Proceed*; the connection is fully encrypted either way. To remove the warning on a particular PC, import `/opt/sspl-admin/certs/sspl-admin.crt` into its trusted store.

The **Setup — install components** card at the top shows what is installed and lets you install the rest, in order:

1. **ERPNext stack** — enter the server IP, HTTP port, MariaDB root password, and Administrator password, then click *Install ERPNext*. This pulls the image, starts the containers, and creates the site (10–20 minutes); progress streams into the job console. The passwords are sent over HTTPS as environment variables and are never written to the log.
2. **Backup system** and **Update & rollback scripts** — enabled once ERPNext is installed, and reuse the deployed site name automatically. One click each.

Each switch becomes an *installed* ✓ status when done.

Day-to-day use

- **Actions** — full backup, DB-only backup, verify, system update, rollback (with a snapshot picker), and a Clear-RAM-caches button. Only one job runs at a time; its output streams into the job console.
- **Backups** — full backups with DB / Files / Private completeness badges, DB-only dumps, image snapshots, and uploads; every file can be downloaded.
- **Upload** — send backup files to the server; they are stored under `/opt/backups/frappe/uploads/` and never executed or restored automatically.

Restore is deliberately not in the panel: it needs the MariaDB root password and overwrites live data, so it stays a console-only operation (section 6).

Managing the service

```
sudo systemctl status sspl-admin      # is it running?
sudo systemctl restart sspl-admin     # restart
sudo journalctl -u sspl-admin -f      # follow the logs
```

To change the password or port, re-run `./setup_admin_panel.sh` — it keeps the certificate and Python environment. Keep port 8090 LAN-only; never port-forward it to the internet.

14. Keeping this toolkit up to date

When this repository gets improvements, refresh everything installed on the server in one command:


```
cd sspl_installation
git pull
./update_tooling.sh
```

It re-installs the backup scripts (`/opt/scripts/v2/`), the update/rollback scripts (`/opt/sspl-erp/v2/`), and the admin panel (`/opt/sspl-admin/` , restarting and health-checking its service). It is fully non-interactive and **preserves all of your configuration**: site name, rclone remote, retention days, alert email, the panel's login, port, and certificate. Cron jobs are not changed, and components that were never installed are skipped with a pointer to the right installer.

15. Quick command reference

I want to...	Run
Set up a brand-new server	<code>cd "Production Installation" && ./install_sspl_erp.sh</code>
Back up everything now	<code>sudo /opt/scripts/v2/frappe_backup.sh</code>
Back up only the database	<code>sudo /opt/scripts/v2/frappe_db_backup.sh</code>
Check my backups are OK	<code>sudo /opt/scripts/v2/frappe_backup_verify.sh</code>
Restore a backup	<code>sudo /opt/scripts/v2/frappe_restore.sh /opt/backups/frappe/<TIMESTAMP></code>
Update to the latest release	<code>cd /opt/sspl-erp/v2 && ./sspl-erp-update-with-rollback.sh</code>
Undo the last update	<code>cd /opt/sspl-erp/v2 && ./sspl-erp-rollback.sh</code>
See image snapshots	<code>./sspl-erp-backup-manager.sh list</code>
Free disk space	<code>./sspl-erp-backup-manager.sh clean 3</code>
Open the admin panel	<code>https://<server-ip>:8090</code>
Restart the admin panel	<code>sudo systemctl restart sspl-admin</code>
Update all installed tooling	<code>git pull && ./update_tooling.sh</code>
Check services	<code>docker compose ps</code>
Check ERPNext version	<code>docker compose exec backend bench version</code>
View recent logs	<code>docker compose logs --tail=50</code>

16. Troubleshooting

Backup verification says something is MISSING

```
sudo tail -50 /var/log/frappe_backup_v2.log      # find the failing step
ls -lh /opt/backups/frappe/<latest>/             # see what was actually saved
sudo /opt/scripts/v2/frappe_backup.sh           # re-run manually and watch the output
```

Update failed partway through

The script prints instructions when it fails. In short: roll back the images with `./sspl-erp-rollback.sh`; if the migration had already modified the database, also restore the data backup taken at the start of the update (section 6).

Rollback cannot find or load a snapshot

```
ls -lh /opt/sspl-erp/image-backups/             # is the .tar there?
tar -tvf /opt/sspl-erp/image-backups/backup_*.tar | head # is it readable?
# Load manually if needed:
docker load -i /opt/sspl-erp/image-backups/backup_<TIMESTAMP>.tar
docker compose up -d
```

Site unreachable after update / rollback

```
docker compose ps                                # any container restarting or exited?
docker compose logs backend --tail=100
docker compose exec backend bench --site 192.168.225.135 doctor
```

Disk is filling up

```
du -sh /opt/sspl-erp/image-backups /opt/backups/frappe
./sspl-erp-backup-manager.sh clean 2              # image snapshots
# Data backup retention: lower RETENTION_DAYS in /opt/scripts/v2/frappe_backup.sh
```

Cron jobs not running

```
sudo crontab -l                                # jobs present?
grep CRON /var/log/syslog | tail                # did they fire?
sudo tail /var/log/frappe_backup_v2.log          # what did they say?
```